US010719761B2

(12) **United States Patent**
Shazeer et al.

(10) **Patent No.: US 10,719,761 B2**
(45) **Date of Patent: Jul. 21, 2020**

(54) **MIXTURE OF EXPERTS NEURAL NETWORKS**

(71) Applicant: **Google LLC**, Mountain View, CA (US)

(72) Inventors: **Noam M. Shazeer**, Palo Alto, CA (US); **Azalia Mirhoseini**, San Jose, CA (US); **Krzysztof Stanislaw Maziarz**, Jaslo (PL)

(73) Assignee: **Google LLC**, Mountain View, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

(21) Appl. No.: **16/393,063**

(22) Filed: **Apr. 24, 2019**

(65) **Prior Publication Data**

US 2019/0251423 A1 Aug. 15, 2019

Related U.S. Application Data

(63) Continuation of application No. PCT/US2017/059909, filed on Nov. 3, 2017.

(60) Provisional application No. 62/432,497, filed on Dec. 9, 2016, provisional application No. 62/418,135, filed on Nov. 4, 2016.

(51) **Int. Cl.**
G06N 3/04 (2006.01)
G06N 3/08 (2006.01)

(52) **U.S. Cl.**
CPC ***G06N 3/0454*** (2013.01); ***G06N 3/08*** (2013.01)

(58) **Field of Classification Search**
CPC G06N 3/02; G06N 3/0454; G06N 3/04
See application file for complete search history.

(56) **References Cited**

PUBLICATIONS

Eigen, David, Marc'Aurelio Ranzato, and Ilya Sutskever, "Learning factored representations in a deep mixture of experts", arXiv preprint arXiv:1312.4314, 2013, 8 pages. (Year: 2013).*
Leonard, Nicholas, "Distributed conditional computing", MSc thesis, University of Montreal, Aug. 2014, 90 pages. (Year: 2014).*
Garmash et al, "Ensemble learning for multi-source neural machine translation," Coling, 2016, 10 pages.
Jordan et al, "Hierarchical mixtures of experts and the EM algorithm," MA Institute of Technology, 1993, 31 pages.
PCT International Search Report and Written Opinion in International Appln. No. PCT/US2017/059909, dated Feb. 7, 2018, 14 pages.

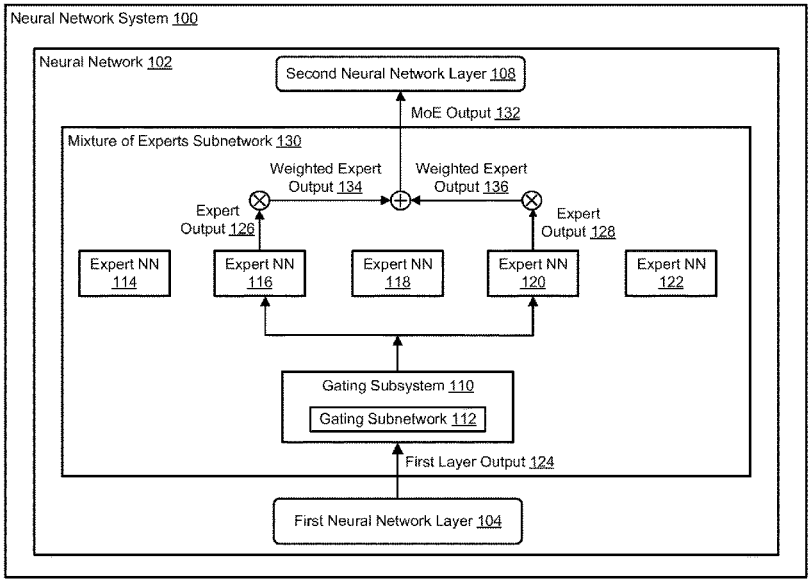
* cited by examiner

Primary Examiner — Vincent Gonzales
(74) *Attorney, Agent, or Firm* — Fish & Richardson P.C.

(57) **ABSTRACT**

A system includes a neural network that includes a Mixture of Experts (MoE) subnetwork between a first neural network layer and a second neural network layer. The MoE subnetwork includes multiple expert neural networks. Each expert neural network is configured to process a first layer output generated by the first neural network layer to generate a respective expert output. The MoE subnetwork further includes a gating subsystem that selects, based on the first layer output, one or more of the expert neural networks and determine a respective weight for each selected expert neural network, provides the first layer output as input to each of the selected expert neural networks, combines the expert outputs generated by the selected expert neural networks in accordance with the weights for the selected expert neural networks to generate an MoE output, and provides the MoE output as input to the second neural network layer.

17 Claims, 3 Drawing Sheets



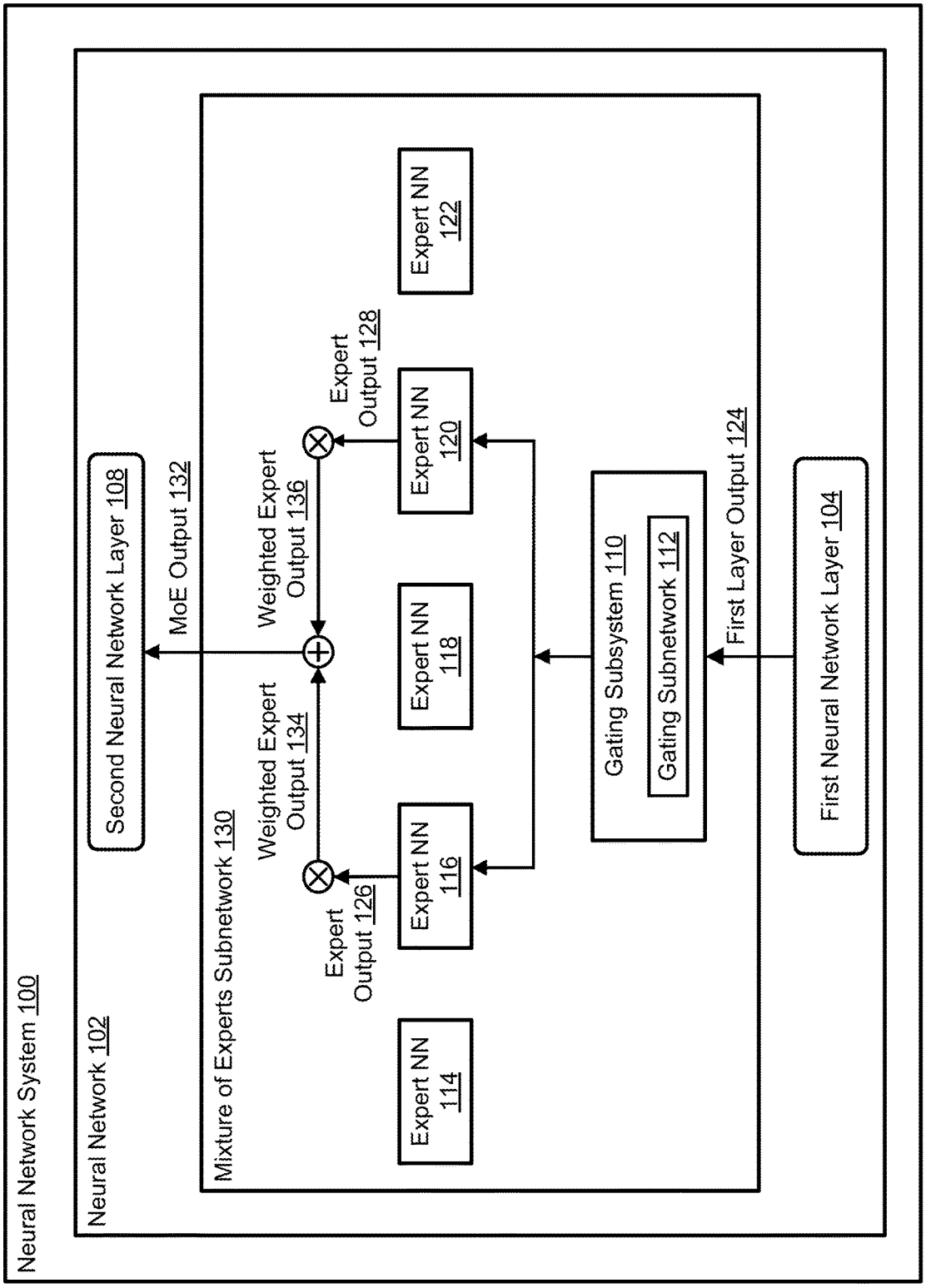

Neural Network System 100
Neural Network 102
Second Neural Network Layer 108
MoE Output 132
Mixture of Experts Subnetwork 130
Weighted Expert Output 136
Weighted Expert Output 134
Expert Output 128
Expert Output 126
Expert NN 114
Expert NN 116
Expert NN 118
Expert NN 120
Expert NN 122
Gating Subsystem 110
Gating Subnetwork 112
First Layer Output 124
First Neural Network Layer 104

FIG. 1

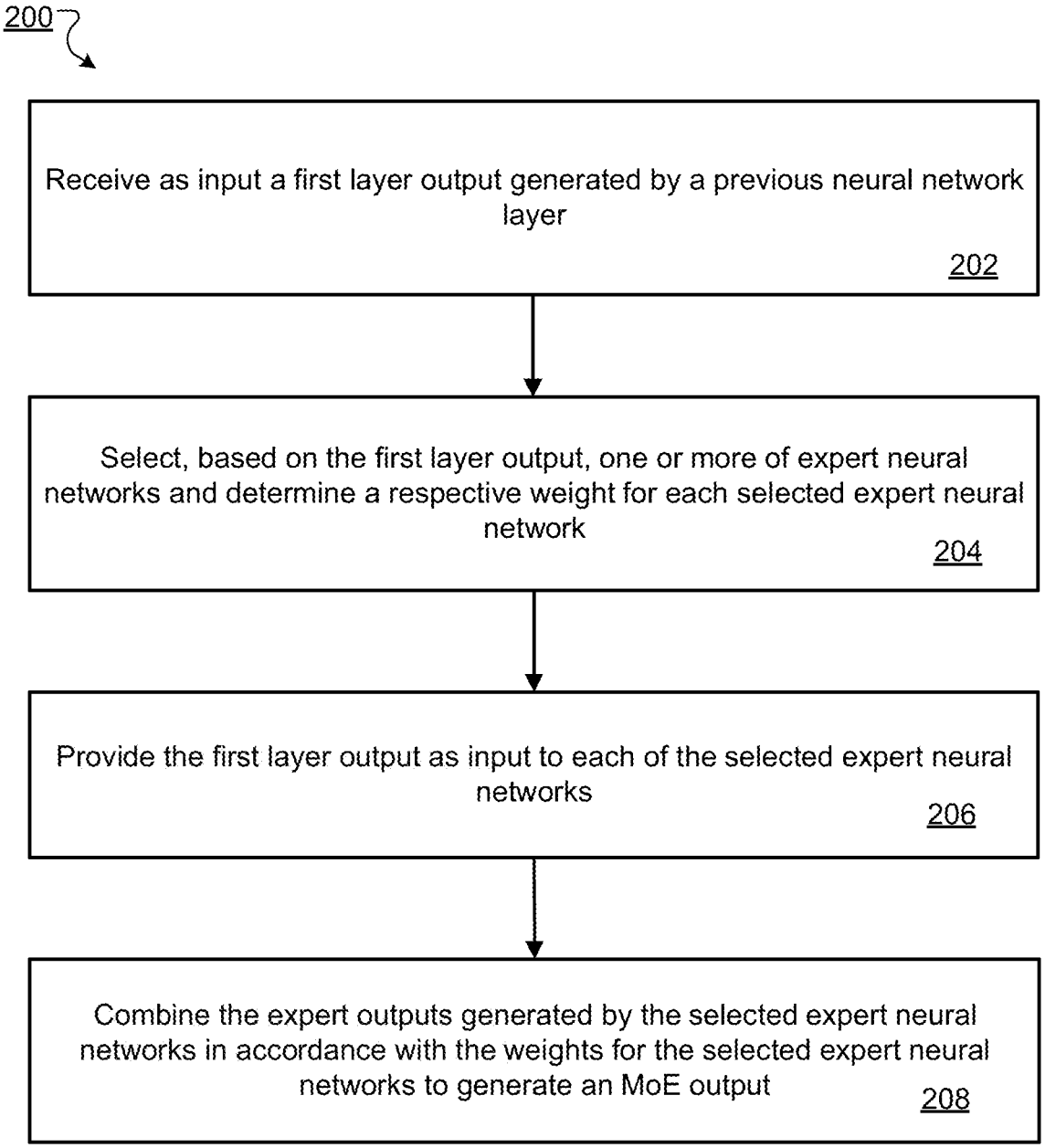

200
Receive as input a first layer output generated by a previous neural network layer
202
Select, based on the first layer output, one or more of expert neural networks and determine a respective weight for each selected expert neural network
204
Provide the first layer output as input to each of the selected expert neural networks
206
Combine the expert outputs generated by the selected expert neural networks in accordance with the weights for the selected expert neural networks to generate an MoE output
208

FIG. 2

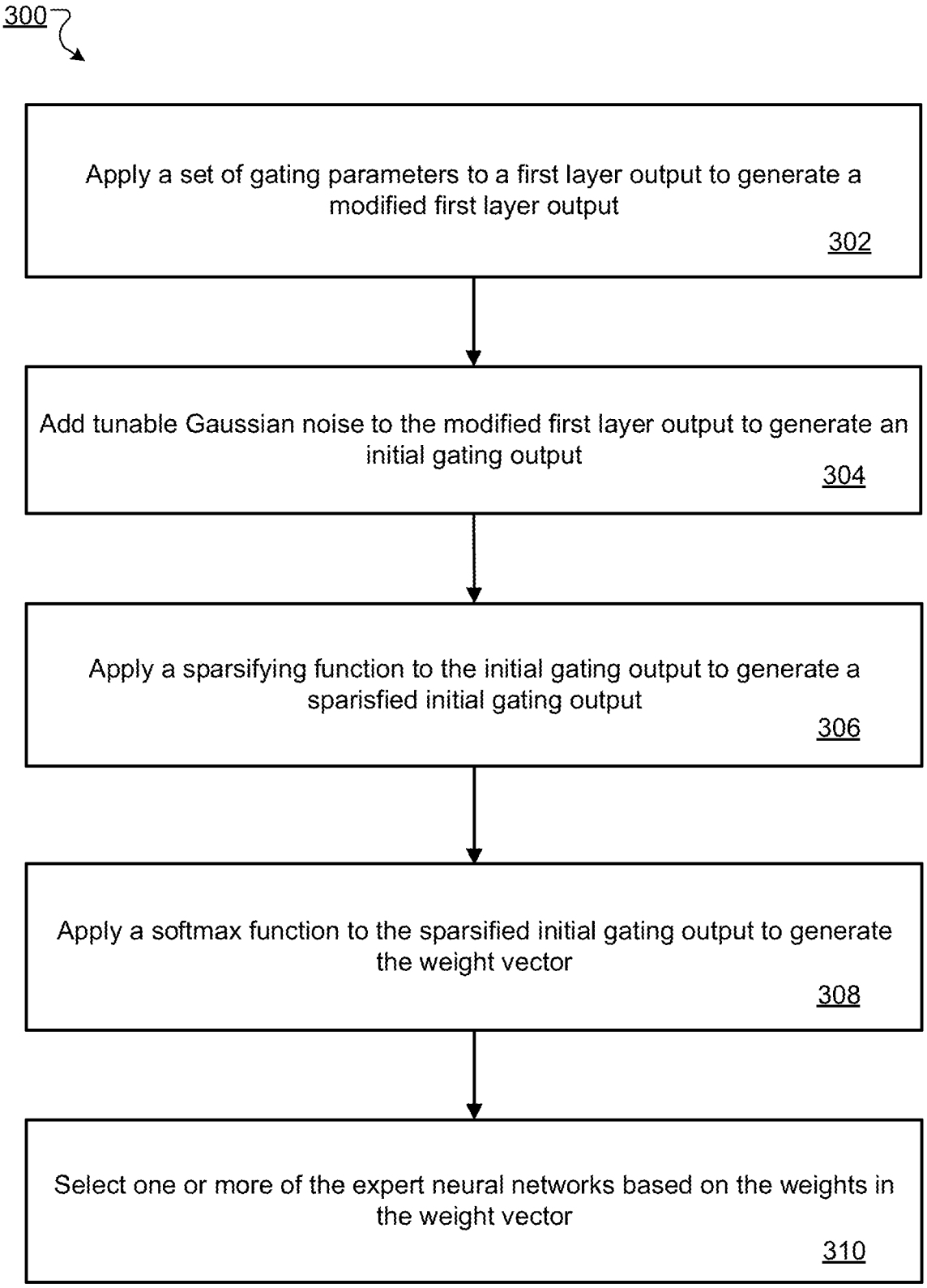

300
Apply a set of gating parameters to a first layer output to generate a modified first layer output
302
Add tunable Gaussian noise to the modified first layer output to generate an initial gating output
304
Apply a sparsifying function to the initial gating output to generate a sparisfied initial gating output
306
Apply a softmax function to the sparsified initial gating output to generate the weight vector
308
Select one or more of the expert neural networks based on the weights in the weight vector
310

FIG. 3

MIXTURE OF EXPERTS NEURAL NETWORKS

CROSS-REFERENCE TO RELATED APPLICATION

This application claims the benefit of International Application No. PCT/US2017/059909, filed on Nov. 3, 2017, which claims priority to U.S. Provisional Application Ser. No. 62/432,497, filed on Dec. 9, 2016 and U.S. Provisional Application Ser. No. 62/418,135, filed on Nov. 4, 2016. The disclosure of these prior applications is considered part of and is incorporated by reference in the disclosure of this application.

BACKGROUND

This specification relates to processing inputs through the layers of neural network to generate outputs.

Neural networks are machine learning models that employ one or more layers of nonlinear units to predict an output for a received input. Some neural networks include one or more hidden layers in addition to an output layer. The output of each hidden layer is used as input to another layer in the network, i.e., another hidden layer or the output layer. Each layer of the network generates an output from a received input in accordance with current values of a respective set of parameters.

SUMMARY

In general, one innovative aspect of the subject matter described in this specification can be embodied in a system implemented as computer programs on one or more computers in one or more locations that includes a neural network. The neural network includes a Mixture of Experts (MoE) subnetwork between a first neural network layer and a second neural network layer in the neural network. The MoE subnetwork includes a plurality of expert neural networks, in which each expert neural network is configured to process a first layer output generated by the first neural network layer in accordance with a respective set of expert parameters of the expert neural network to generate a respective expert output. The MoE subnetwork further includes a gating subsystem configured to: select, based on the first layer output, one or more of the expert neural networks and determine a respective weight for each selected expert neural network, provide the first layer output as input to each of the selected expert neural networks, combine the expert outputs generated by the selected expert neural networks in accordance with the weights for the selected expert neural networks to generate an MoE output, and provide the MoE output as input to the second neural network layer.

The foregoing and other embodiments can each optionally include one or more of the following features, alone or in combination. The expert neural networks may have the same or similar architectures but different parameter values. Combining the expert outputs generated by the selected expert neural network may include: weighting the expert output generated by each of the selected expert neural networks by the weight for the selected expert neural network to generate a weighted expert output, and summing the weighted expert outputs to generate the MoE output. The gating subsystem may include a gating subnetwork, in which the gating subnetwork is configured to: process the first layer output to generate a weight vector that includes a respective weight for each of the plurality of expert neural networks in accordance with a set of gating parameters, and select one or more of the expert neural networks based on the weights in the weight vector. The weight vector may be a sparse vector that includes non-zero weights for only a few of the expert neural networks. Selecting one or more the expert neural networks may include selecting only expert neural networks that have non-zero weights in the weight vector.

Processing the first layer output to generate a weight vector that includes a respective weight for each of the plurality of expert neural networks in accordance with a set of gating parameters may include: generating an initial gating output, wherein generating the initial gating output comprises applying the set of gating parameters to the first layer output; applying a sparsifying function to the initial gating output to generate a sparsified initial gating output; and applying a softmax function to the sparsified initial gating output to generate the weight vector.

Generating the initial gating output may include: applying the set of gating parameters to the first layer output to generate a modified first layer output; and adding tunable Gaussian noise to the modified first layer output to generate the initial gating output. Adding tunable Gaussian noise to the modified first layer output to generate the initial gating output may include: applying a set of trainable noise parameters to the first layer output to generate an initial noise output; element-wise multiplying the initial noise output by a vector of noise values sampled from a normal distribution to generate a final noise output; and adding the final noise output to the modified first layer output. The sparsifying function sets all values in the initial gating output other than the k highest values to a value that is mapped to zero by the softmax function. The gating subsystem comprises a parent gating subnetwork and a plurality of child gating subnetworks, in which each of the child gating subnetworks manages a disjoint subset of the plurality of expert neural networks from each other child gating subnetwork.

Another innovative aspect of the subject matter described in this specification can be embodied in a method including: receiving a network input; and processing the network input using the system described above to generate a network output for the network input.

Other innovative aspects of the subject matter described in this specification can be embodied in one or more non-transitory storage media encoded with instructions that when implemented by one or more computers cause the one or more computers to implement the system and method described above.

The subject matter described in this specification can be implemented in particular embodiments so as to realize one or more of the following advantages. While some MoE implementations involve significant training time and computational costs, various embodiments described herein relate to MoE implementations which improve computational capacity without greatly increasing computing resource requirements. In particular, various embodiments described in this specification relate to MoE implementations in which computational capacity (number of parameters) of a neural network that includes a MoE subnetwork can be increased by leveraging the availability of a large number of expert neural networks. In some MoE implementations described herein, only a small number of the expert neural networks are selected during the processing of any given network input by the neural network, and hence the processing time and computing resources necessary to process an inference using the neural network can be maintained at a reasonable level. That is, computational capacity

can be greatly increased while maintaining a reasonable processing time and without greatly increasing how many computing resources are required to process an inference. As a result, the neural network can achieve significantly better results (e.g., better prediction accuracy) than prior approaches at reasonable processing time and computational costs, especially for tasks where model capacity is critical for absorbing the vast quantities of knowledge available in the training dataset. Therefore, the subject matter described in this specification may be implemented in particular embodiments to achieve various technical benefits. Moreover, neural network systems in accordance with various embodiments may be applied for a variety of technical purposes, including but not limited to estimating the likelihood that an image contains an image of an object belonging to a category.

The details of one or more embodiments of the subject matter of this specification are set forth in the accompanying drawings and the description below. Other features, aspects, and advantages of the subject matter will become apparent from the description, the drawings, and the claims.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. **1** is a block diagram of an example of a neural network system.

FIG. **2** is a flow diagram of an example process for generating a mixture-of-experts (MoE) output for an input.

FIG. **3** is a flow diagram of an example process for determining a weight vector and for selecting one or more of the expert neural networks based on the weight vector.

Like reference numbers and designations in the various drawings indicate like elements.

DETAILED DESCRIPTION

FIG. **1** shows an example of a neural network system **100**. The neural network system **100** is an example of a system implemented as computer programs on one or more computers in one or more locations, in which the systems, components, and techniques described below can be implemented. Generally, the system **100** includes a neural network **102** that can be configured to receive any kind of digital data input and to generate any kind of score, classification, or regression output based on the input.

For example, if the inputs to the neural network **102** are images or features that have been extracted from images, the output generated by the neural network **102** for a given image may be scores for each of a set of object categories, with each score representing an estimated likelihood that the image contains an image of an object belonging to the category.

As another example, if the inputs to the neural network **102** are Internet resources (e.g., web pages), documents, or portions of documents or features extracted from Internet resources, documents, or portions of documents, the output generated by the neural network for a given Internet resource, document, or portion of a document may be a score for each of a set of topics, with each score representing an estimated likelihood that the Internet resource, document, or document portion is about the topic.

As another example, if the inputs to the neural network **102** are features of an impression context for a particular content element, the output generated by the neural network **102** may be a score that represents an estimated likelihood that the particular content element will be clicked on.

As another example, if the inputs to the neural network **102** are features of a personalized recommendation for a user, e.g., features characterizing the context for the recommendation, e.g., features characterizing previous actions taken by the user, the output generated by the neural network **102** may be a score for each of a set of content elements, with each score representing an estimated likelihood that the user will respond favorably to being recommended the content element.

As another example, if the input to the neural network **102** is a sequence of text in one language, the output generated by the neural network **102** may be a score for each of a set of pieces of text in another language, with each score representing an estimated likelihood that the piece of text in the other language is a proper translation of the input text into the other language.

As another example, if the input to the neural network **102** is a sequence representing a spoken utterance, the output generated by the neural network **102** may be a score for each of a set of pieces of text, each score representing an estimated likelihood that the piece of text is the correct transcript for the utterance.

The neural network **102** includes a stack of neural network layers. The stack of neural network layers can be any kind of neural network layers, for example, Long Short-Term Memory (LSTM) neural network layers or other recurrent neural network layers, convolutional neural network layers, or fully-connected neural network layers. In some implementations, the stack of neural network layers may include a mixture of two or more different kinds of neural network layers. To increase the computational capacity of the neural network **102**, the neural network **102** includes one or more Mixture of Experts (MoE) subnetworks between the stacked neural network layers.

In particular, as shown in FIG. **1**, the neural network **102** includes a MoE subnetwork **130** arranged between a first neural network layer **104** and a second neural network layer **108** in the neural network **102**. The first neural network layer **104** and the second neural network layer **108** can be any kind of neural network layer, for example, a LSTM neural network layer or other recurrent neural network layer, a convolutional neural network layer, or a fully-connected neural network layer. In some implementations, one or both of the first neural network layer **104** and the second neural network layer **108** can be other MoE subnetworks.

The MoE subnetwork **130** includes multiple expert neural networks, e.g., expert neural networks **114-122**. Each expert neural network in the MoE subnetwork **130** can be configured to process a first layer output **124** generated by the first neural network layer **104** in accordance with a respective set of expert parameters of the expert neural network to generate a respective expert output.

While FIG. **1**. shows that the MoE subnetwork **130** includes only five expert neural networks, in general, a MoE subnetwork can include a large number of expert neural networks, e.g., thousands of expert neural networks. Each expert neural network can be a feed-forward neural network with its own parameters. The expert neural networks are configured to receive the same sized inputs and produce the same-sized outputs. In some implementations, the expert neural networks are feed-forward neural networks with identical architectures, but with different parameters. Each expert neural network may have a very large number of parameters. For example, each expert neural network may have one to two million parameters, making the total number of parameters of the MoE subnetwork to billions of parameters.

Although the MoE subnetwork includes a large number of expert neural networks, only a small number of them are selected during the processing of any given network input by the neural network **102**, e.g., only a small number of the expert neural networks are selected to process the first layer output **124**. The neural network **102** includes a gating subsystem **110** that is configured to select, based on the first layer output **124**, one or more of the expert neural networks, and determine a respective weight for each selected expert neural network. The gating subsystem **110** then provides the first layer output as input to each of the selected expert neural networks. The gating subsystem **110** combines the expert outputs generated by the selected expert neural networks in accordance with the weights for the selected expert neural networks to generate a MoE output **132**. The gating subsystem **110** then provides the MoE output **132** as input to the second neural network layer **108**.

In particular, the gating subsystem **110** includes a gating subnetwork **112** that is configured to process the first layer output to generate a weight vector that includes a respective weight for each of the plurality of expert neural networks in accordance with a set of gating parameters of the gating subnetwork, and to select one or more of the expert neural networks based on the weights in the weight vector.

For example, assuming the MoE subnetwork **130** includes a set of n expert neural networks $E_1, \ldots, E_n$, the gating subnetwork **112**, denoted as G, is configured to process a given input x (e.g., the first layer output **124**) to generate a weight vector G(x). The weight vector G(x) includes a respective weight for each expert neural network in the set of n expert neural networks. The gating subnetwork **112** is configured to select one or more expert neural networks from the set of n expert neural networks based on the weights in the weight vector G(x). In particular, the weight vector G(x) is a sparse n-dimensional vector that includes non-zero weights for only a few of the expert neural networks in the set. The number of non-zero weights is an integer and is very small in comparison with the number of expert neural networks in the MoE subnetwork **130**. For example, the MoE subnetwork **130** may have hundreds to thousands of expert neural networks, and the weight vector G(x) may have one or two non-zero weights. The number of non-zero weights can be determined based on criteria such as computational costs, model quality (e.g., prediction quality), and whether the MoE subnetwork **130** is under a testing or training condition. Generally, smaller numbers of non-zero weights require less computation while larger numbers of non-zero weights lead to higher model quality. The number of non-zero weights is larger than 1 in order to train the MoE subnetwork **130**. The gating subnetwork **112** selects only expert neural networks that have non-zero weights in the weight vector G(x). For example, the expert neural networks **116** and **120** in FIG. **1** have non-zero weights and therefore are selected for further processing.

The process for generating the weight vector with a pre-determined number of non-zero weights is described in more detail below with reference to FIG. **3**.

After the weight vector has been generated and one or more expert neural networks (e.g., expert neural networks **116** and **120**) have been selected, the gating subsystem **110** combines the expert outputs generated by the selected expert neural networks by weighting the expert output generated by each of the selected expert neural networks (e.g., expert output **126** generated by the expert neural network **116** and expert output **128** generated by the expert neural network **120**) by the weight for the selected expert neural network to generate a weighted expert output, and summing the weighted expert outputs (e.g., the weighted expert outputs **134** and **136**) to generate the MoE output **132**.

In particular, the MoE output **132** can be expressed as follows:

$$y = \sum_{i=1}^{n} G(x)_i E_i(x),$$

where y is the MoE output that is computed based on the sparsity of the output of the weight vector G(x). That means wherever $G(x)_i=0$, the i-th expert neural network E_i in the set of expert neural networks does not need to compute the respective expert output $E_i(x)$.

After generating the MoE output **132**, the gating subsystem **110** provides the MoE output **132** as input to the second neural network layer **108** for further processing.

In some implementations, the second neural network layer **108** can be another MoE subnetwork instead of a conventional neural network layer.

In some implementations, if the number of expert neural networks is very large (for example, more than one thousand expert neural networks), the branching factor of the gating subnetwork **112** can be reduced by using a multi-level hierarchical MoE subnetwork. For example, in a two-level hierarchical MoE subnetwork, the gating subsystem **110** can include a parent gating subnetwork and multiple child gating subnetworks, in which each of the child gating subnetworks is itself a secondary MoE subnetwork with its own gating subnetwork. Each of the child gating subnetworks manages a disjoint subset of multiple expert neural networks from each other child gating subnetwork. The parent gating subnetwork chooses a sparse weighted combination of the child gating subnetworks and generates a MoE output based on the sparse weighted combination.

The use of conditional computation, where parts of the MoE subnetwork **130** are active on a per-input example basis, can significantly increase the computational capacity of the neural network **102** without a proportional increase in computation. This is because the MoE subnetwork **130** can have thousands of expert neural networks but only very few expert neural networks need to be evaluated for each input. As a result, the neural network **102** can achieve significantly better results than prior approaches at lower computational costs, especially for tasks such as language modeling and machine translation, where model capacity is critical for absorbing the vast quantities of knowledge available in the training corpora. As an example, the neural network **102** can be trained on a language modeling task that has a dataset including a billion to a hundred billion words. In another example, the neural network **102** can be trained on a single language pair machine translation task with a dataset of tens of millions of sentence pairs. In yet another example, the neural network **102** can be trained on a multilingual machine translation task that has a very large combined dataset of twelve language pairs. During training, various components of the neural network **102** can be trained jointly by back-propagation.

FIG. **2** is a flow diagram of an example process **200** for generating a mixture-of-experts (MoE) output for an input using an MoE subnetwork, e.g., the MoE subnetwork **130** of FIG. **1**. For convenience, the process will be described as being performed by a system of one or more computers located in one or more locations. For example, a neural network system, e.g., the neural network system **100** of FIG. **1**, appropriately programmed, can perform the process.

The system receives as input a first layer output generated by a neural network layer preceding the MoE subnetwork (step **202**). The MoE subnetwork includes multiple expert neural networks and a gating subsystem that includes a gating subnetwork.

The system selects, based on the first layer output, one or more of the multiple expert neural networks and determines a respective weight for each selected expert neural network (step **204**).

In particular, the system processes the first layer output to generate a weight vector that includes a respective weight for each of the multiple expert neural networks in accordance with a set of gating parameters of the gating subnetwork. The system then selects one or more of the multiple expert neural networks based on the weights in the weight vector. Specifically, the weight vector is a sparse vector that includes non-zero weights for only a few of the expert neural networks, and the system selects only expert neural networks that have non-zero weights in the weight vector. Generating the weight vector is described in more detail below with reference to FIG. **3**.

The system provides the first layer output as input to each of the selected expert neural networks (step **206**). Each of the selected expert neural networks is configured to process the first layer output in accordance with the respective current values of parameters of the selected expert neural network to generate a respective expert output.

The system then combines the expert outputs generated by the selected expert neural networks in accordance with the weights for the selected expert neural networks to generate the MoE output (step **208**).

In particular, the system weights the expert output generated by each of the selected expert neural networks by the weight for the selected expert neural network to generate a weighted expert output. The system then sums the weighted expert outputs to generate the MoE output.

The system then provides the MoE output as input to a second neural network layer following the MoE subnetwork for further processing.

FIG. **3** is a flow diagram of an example process **300** for determining a weight vector and for selecting one or more of the expert neural networks included in a MoE subnetwork, e.g., the MoE subnetwork **130** of FIG. **1**, based on the weight vector. For convenience, the process will be described as being performed by a system of one or more computers located in one or more locations. For example, a neural network system, e.g., the neural network system **100** of FIG. **1**, appropriately programmed, can perform the process.

The system applies a set of gating parameters of the gating subnetwork of the MoE subnetwork to a first layer output to generate a modified first layer output (step **302**).

The system adds a noise to the modified first layer output to generate an initial gating output (step **304**). The noise helps with load balancing, i.e., to encourage expert neural networks to receive roughly equal numbers of training examples in a training dataset during training, or to receive roughly equal numbers of input examples in input data during testing.

For example, the noise can be a tunable Gaussian noise. To add a tunable Gaussian noise to the modified first layer output, the system first applies a set of trainable noise parameters to the first layer output to generate an initial noise output. The system then element-wise multiplies the initial noise output by a vector of noise values sampled from a normal distribution to generate a final noise output. In some implementations, the system may apply a softplus function on the initial noise output before element-wise multiplying the initial noise output by the vector of noise values sampled from the normal distribution to generate the final noise output. The system then adds the final noise output to the modified first layer output to generate the initial gating output.

For example, the initial gating output for the first layer output x can be denoted as a vector H(x), which has a dimension equal to the number of expert neural network in the MoE subnetwork. The i-th element of H(x) that corresponds to the i-th expert neural network can be computed as follows:

$$H(x)_i=(x\cdot W_g)_i+\text{StandardNormal}(\ )\cdot\text{Softplus}((x\cdot W_{noise})_i)$$

where W_g is the set of gating parameters that is applied to the first layer output x to generate a modified first layer output x W_g; W_{noise} is the set of trainable noise parameters; and StandardNormal( ) is a normal distribution that is used to sample the vector of noise values.

The system applies a sparsifying function to the initial gating output to generate a sparsified initial gating output (step **306**). The sparsifying function sets all values in the initial gating output other than the k highest values to a value that is mapped to zero by a softmax function, where k is an integer. k can be determined based on criteria such as computational costs, model quality (e.g., prediction quality), and whether the MoE subnetwork is under a testing or training condition. Generally, smaller values of k require less computation while larger values of k lead to higher model quality. k is larger than 1 in order to train the MoE subnetwork. An example of the sparsifying function has the following form:

$$KeepTopK(v,k)_i=\begin{cases} v_i & \text{if } v_i \text{ is in the top } k \text{ elements of } v \\ -\infty & \text{otherwise.} \end{cases}.$$

The sparsified initial gating output can be expressed as KeepTopK(H(x), k).

The system applies the softmax function to the sparsified initial gating output to generate a weight vector (step **308**). The weight vector is a sparse vector that includes non-zero weights for only a few of the expert neural networks. For example, the weight vector G(x) can be computed as follows:

$$G(x)=\text{Softmax}(\text{KeepTop}K(H)x),k)).$$

The system selects one or more of the expert neural networks based on the weights in the weight vector (step **310**). Specifically, the system selects only expert neural networks that have non-zero weights in the weight vector.

During training of the neural network **102**, the processes **200** and **300** can be used as part of generating a network output for a training input. The gradient of an objective function can be backpropagated to adjust the values of the parameters of various components of the neural network **102** to improve the quality of the network output relative to a known output for the training input.

This specification uses the term "configured" in connection with systems and computer program components. For a system of one or more computers to be configured to perform particular operations or actions means that the system has installed on it software, firmware, hardware, or a combination of them that in operation cause the system to perform the operations or actions. For one or more computer programs to be configured to perform particular operations

or actions means that the one or more programs include instructions that, when executed by data processing apparatus, cause the apparatus to perform the operations or actions.

Embodiments of the subject matter and the functional operations described in this specification can be implemented in digital electronic circuitry, in tangibly-embodied computer software or firmware, in computer hardware, including the structures disclosed in this specification and their structural equivalents, or in combinations of one or more of them. Embodiments of the subject matter described in this specification can be implemented as one or more computer programs, i.e., one or more modules of computer program instructions encoded on a tangible non-transitory storage medium for execution by, or to control the operation of, data processing apparatus. The computer storage medium can be a machine-readable storage device, a machine-readable storage substrate, a random or serial access memory device, or a combination of one or more of them. Alternatively, or in addition, the program instructions can be encoded on an artificially-generated propagated signal, e.g., a machine-generated electrical, optical, or electromagnetic signal, that is generated to encode information for transmission to suitable receiver apparatus for execution by a data processing apparatus.

The term "data processing apparatus" refers to data processing hardware and encompasses all kinds of apparatus, devices, and machines for processing data, including by way of example a programmable processor, a computer, or multiple processors or computers. The apparatus can also be, or further include, special purpose logic circuitry, e.g., an FPGA (field programmable gate array) or an ASIC (application-specific integrated circuit). The apparatus can optionally include, in addition to hardware, code that creates an execution environment for computer programs, e.g., code that constitutes processor firmware, a protocol stack, a database management system, an operating system, or a combination of one or more of them.

A computer program, which may also be referred to or described as a program, software, a software application, an app, a module, a software module, a script, or code, can be written in any form of programming language, including compiled or interpreted languages, or declarative or procedural languages; and it can be deployed in any form, including as a stand-alone program or as a module, component, subroutine, or other unit suitable for use in a computing environment. A program may, but need not, correspond to a file in a file system. A program can be stored in a portion of a file that holds other programs or data, e.g., one or more scripts stored in a markup language document, in a single file dedicated to the program in question, or in multiple coordinated files, e.g., files that store one or more modules, sub-programs, or portions of code. A computer program can be deployed to be executed on one computer or on multiple computers that are located at one site or distributed across multiple sites and interconnected by a data communication network.

The processes and logic flows described in this specification can be performed by one or more programmable computers executing one or more computer programs to perform functions by operating on input data and generating output. The processes and logic flows can also be performed by special purpose logic circuitry, e.g., an FPGA or an ASIC, or by a combination of special purpose logic circuitry and one or more programmed computers.

Computers suitable for the execution of a computer program can be based on general or special purpose microprocessors or both, or any other kind of central processing unit. Generally, a central processing unit will receive instructions and data from a read-only memory or a random access memory or both. The essential elements of a computer are a central processing unit for performing or executing instructions and one or more memory devices for storing instructions and data. The central processing unit and the memory can be supplemented by, or incorporated in, special purpose logic circuitry. Generally, a computer will also include, or be operatively coupled to receive data from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto-optical disks, or optical disks. However, a computer need not have such devices. Moreover, a computer can be embedded in another device, e.g., a mobile telephone, a personal digital assistant (PDA), a mobile audio or video player, a game console, a Global Positioning System (GPS) receiver, or a portable storage device, e.g., a universal serial bus (USB) flash drive, to name just a few.

Computer-readable media suitable for storing computer program instructions and data include all forms of non-volatile memory, media and memory devices, including by way of example semiconductor memory devices, e.g., EPROM, EEPROM, and flash memory devices; magnetic disks, e.g., internal hard disks or removable disks; magneto-optical disks; and CD-ROM and DVD-ROM disks.

To provide for interaction with a user, embodiments of the subject matter described in this specification can be implemented on a computer having a display device, e.g., a CRT (cathode ray tube) or LCD (liquid crystal display) monitor, for displaying information to the user and a keyboard and a pointing device, e.g., a mouse or a trackball, by which the user can provide input to the computer. Other kinds of devices can be used to provide for interaction with a user as well; for example, feedback provided to the user can be any form of sensory feedback, e.g., visual feedback, auditory feedback, or tactile feedback; and input from the user can be received in any form, including acoustic, speech, or tactile input. In addition, a computer can interact with a user by sending documents to and receiving documents from a device that is used by the user; for example, by sending web pages to a web browser on a user's device in response to requests received from the web browser. Also, a computer can interact with a user by sending text messages or other forms of message to a personal device, e.g., a smartphone that is running a messaging application, and receiving responsive messages from the user in return.

Data processing apparatus for implementing machine learning models can also include, for example, special-purpose hardware accelerator units for processing common and compute-intensive parts of machine learning training or production, i.e., inference, workloads.

Machine learning models can be implemented and deployed using a machine learning framework, e.g., a TensorFlow framework, a Microsoft Cognitive Toolkit framework, an Apache Singa framework, or an Apache MXNet framework.

Embodiments of the subject matter described in this specification can be implemented in a computing system that includes a back-end component, e.g., as a data server, or that includes a middleware component, e.g., an application server, or that includes a front-end component, e.g., a client computer having a graphical user interface, a web browser, or an app through which a user can interact with an implementation of the subject matter described in this specification, or any combination of one or more such back-end, middleware, or front-end components. The components of

the system can be interconnected by any form or medium of digital data communication, e.g., a communication network. Examples of communication networks include a local area network (LAN) and a wide area network (WAN), e.g., the Internet.

The computing system can include clients and servers. A client and server are generally remote from each other and typically interact through a communication network. The relationship of client and server arises by virtue of computer programs running on the respective computers and having a client-server relationship to each other. In some embodiments, a server transmits data, e.g., an HTML page, to a user device, e.g., for purposes of displaying data to and receiving user input from a user interacting with the device, which acts as a client. Data generated at the user device, e.g., a result of the user interaction, can be received at the server from the device.

While this specification contains many specific implementation details, these should not be construed as limitations on the scope of any invention or on the scope of what may be claimed, but rather as descriptions of features that may be specific to particular embodiments of particular inventions. Certain features that are described in this specification in the context of separate embodiments can also be implemented in combination in a single embodiment. Conversely, various features that are described in the context of a single embodiment can also be implemented in multiple embodiments separately or in any suitable subcombination. Moreover, although features may be described above as acting in certain combinations and even initially be claimed as such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a subcombination or variation of a subcombination.

Similarly, while operations are depicted in the drawings and recited in the claims in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. In certain circumstances, multitasking and parallel processing may be advantageous. Moreover, the separation of various system modules and components in the embodiments described above should not be understood as requiring such separation in all embodiments, and it should be understood that the described program components and systems can generally be integrated together in a single software product or packaged into multiple software products.

Particular embodiments of the subject matter have been described. Other embodiments are within the scope of the following claims. For example, the actions recited in the claims can be performed in a different order and still achieve desirable results. As one example, the processes depicted in the accompanying figures do not necessarily require the particular order shown, or sequential order, to achieve desirable results. In some cases, multitasking and parallel processing may be advantageous.

What is claimed is:

1. A system comprising:

a main neural network implemented by one or more computers, the main neural network comprising a Mixture of Experts (MoE) subnetwork between a first neural network layer and a second neural network layer in the main neural network, wherein the MoE subnetwork comprises:

a plurality of expert neural networks, wherein each expert neural network is configured to process a first layer output generated by the first neural network layer in accordance with a respective set of expert parameters of the expert neural network to generate a respective expert output, and

a gating subsystem configured to:

generate an initial gating output by applying a set of gating parameters to the first layer output,

apply a sparsifying function to the initial gating output to generate a sparsified initial gating output,

apply a softmax function to the sparsified initial gating output to generate a weight vector that includes a respective weight for each of the plurality of expert neural networks,

select, based on the weights in the weight vector, one or more of the expert neural networks and determine a respective weight for each selected expert neural network,

provide the first layer output as input to each of the selected expert neural networks,

combine the expert outputs generated by the selected expert neural networks in accordance with the weights for the selected expert neural networks to generate an MoE output, and

provide the MoE output as input to the second neural network layer.

2. The system of claim **1**, wherein the expert neural networks have the same or similar architectures but different parameter values.

3. The system of claim **1**, wherein combining the expert outputs generated by the selected expert neural network comprises:

weighting the expert output generated by each of the selected expert neural networks by the weight for the selected expert neural network to generate a weighted expert output, and

summing the weighted expert outputs to generate the MoE output.

4. The system of claim **1**, wherein the weight vector is a sparse vector that includes non-zero weights for only a few of the expert neural networks.

5. The system of claim **1**, wherein selecting one or more of the expert neural networks comprises:

selecting only expert neural networks that have non-zero weights in the weight vector.

6. The system of claim **1**, wherein generating the initial gating output comprises:

applying the set of gating parameters to the first layer output to generate a modified first layer output; and

adding tunable Gaussian noise to the modified first layer output to generate the initial gating output.

7. The system of claim **6**, wherein adding tunable Gaussian noise to the modified first layer output to generate the initial gating output comprises:

applying a set of trainable noise parameters to the first layer output to generate an initial noise output;

element-wise multiplying the initial noise output by a vector of noise values sampled from a normal distribution to generate a final noise output; and

adding the final noise output to the modified first layer output.

8. The system of claim **1**, wherein the sparsifying function sets all values in the initial gating output other than the k highest values to a value that is mapped to zero by the softmax function.

9. The system of claim **1**, wherein the gating subsystem comprises a parent gating subnetwork and a plurality of

child gating subnetworks, and wherein each of the child gating subnetworks manages a disjoint subset of the plurality of expert neural networks from each other child gating subnetwork.

10. A method comprising:

receiving a network input; and

processing the network input using a main neural network to generate a network output for the network input, wherein the main neural network is implemented by one or more computers and comprises a Mixture of Experts (MoE) subnetwork between a first neural network layer and a second neural network layer in the main neural network, wherein the network input is a first layer output generated by the first neural network layer preceding the MOE subnetwork, wherein the MoE subnetwork comprises:

a plurality of expert neural networks, wherein each expert neural network is configured to process a first layer output generated by the first neural network layer in accordance with a respective set of expert parameters of the expert neural network to generate a respective expert output, and

wherein processing the network input using the main neural network comprises:

generating an initial gating output by applying a set of gating parameters to the first layer output,

applying a sparsifying function to the initial gating output to generate a sparsified initial gating output,

applying a softmax function to the sparsified initial gating output to generate a weight vector that includes a respective weight for each of the plurality of expert neural networks,

selecting, based on the weights in the weight vector, one or more of the expert neural networks and determining a respective weight for each selected expert neural network,

providing the first layer output as input to each of the selected expert neural networks,

combining the expert outputs generated by the selected expert neural networks in accordance with the weights for the selected expert neural networks to generate an MoE output, and

providing the MoE output as input to the second neural network layer.

11. The method of claim **10**, wherein combining the expert outputs generated by the selected expert neural network comprises:

weighting the expert output generated by each of the selected expert neural networks by the weight for the selected expert neural network to generate a weighted expert output, and

summing the weighted expert outputs to generate the MoE output.

12. The method of claim **10**, wherein the weight vector is a sparse vector that includes non-zero weights for only a few of the expert neural networks.

13. The method of claim **10**, wherein selecting one or more the expert neural networks comprises:

selecting only expert neural networks that have non-zero weights in the weight vector.

14. The method of claim **10**, wherein generating the initial gating output comprises:

applying the set of gating parameters to the first layer output to generate a modified first layer output; and

adding tunable Gaussian noise to the modified first layer output to generate the initial gating output.

15. The method of claim **14**, wherein adding tunable Gaussian noise to the modified first layer output to generate the initial gating output comprises:

applying a set of trainable noise parameters to the first layer output to generate an initial noise output;

element-wise multiplying the initial noise output by a vector of noise values sampled from a normal distribution to generate a final noise output; and

adding the final noise output to the modified first layer output.

16. The method of claim **10**, wherein the sparsifying function sets all values in the initial gating output other than the k highest values to a value that is mapped to zero by the softmax function.

17. One or more computer storage media storing instructions that, when executed by one or more computers, cause the one or more computers to perform operations comprising:

receiving a network input; and

processing the network input using a main neural network to generate a network output for the network input, wherein the main neural network is implemented by one or more computers and comprises a Mixture of Experts (MoE) subnetwork between a first neural network layer and a second neural network layer in the main neural network, wherein the network input is a first layer output generated by the first neural network layer preceding the MOE subnetwork, wherein the MoE subnetwork comprises:

a plurality of expert neural networks, wherein each expert neural network is configured to process a first layer output generated by the first neural network layer in accordance with a respective set of expert parameters of the expert neural network to generate a respective expert output, and

wherein processing the network input using the main neural network comprises:

generating an initial gating output by applying a set of gating parameters to the first layer output,

applying a sparsifying function to the initial gating output to generate a sparsified initial gating output,

applying a softmax function to the sparsified initial gating output to generate a weight vector that includes a respective weight for each of the plurality of expert neural networks,

selecting, based on the weights in the weight vector, one or more of the expert neural networks and determining a respective weight for each selected expert neural network,

providing the first layer output as input to each of the selected expert neural networks,

combining the expert outputs generated by the selected expert neural networks in accordance with the weights for the selected expert neural networks to generate an MoE output, and

providing the MoE output as input to the second neural network layer.

* * * * *